AI 533 **Intelligent Agents and Decision Making** Winter 2025

OREGON STATE UNIVERSITY
School of Electrical Engineering and Computer Science
Instructor: Sandhya Saisubramanian
Submitted by: Manogna Challoju

Total points: 100 Assignment 2: Monte Carlo, TD methods, and Actor-Critic Due date: Feb 24, 2025

Instructions: Collaboration is not allowed on any part of this assignment. It is acceptable to discuss concepts or clarify questions but you must document who you worked with for this assignment. Copying answers or reusing solutions from individuals/the internet is unacceptable and will be dealt with strictly. Solutions must be typed (hand written and scanned submissions will not be accepted) and saved as a .pdf file.

1. **(10 points)** After applying SARSA, can we estimate V^π upon its termination, based on the information available to the agent? If yes, write the equation to estimate V^π . If no, explain why. Yes, after running SARSA, we can estimate $V^\pi(s)$ once it has converged. SARSA is an on-policy learning algorithm, meaning it updates Q-values $Q^\pi(s,a)$ while following a policy π. Since the value function $V^\pi(s)$ represents how good a state is under π, we can compute it using the learned Q-values.

   The relationship between $V^\pi(s)$ and $Q^\pi(s,a)$ is given by:

   $$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$$

   where:

   - $\pi(a|s)$ is the probability of taking action a in state s under policy π.
   - $Q^\pi(s,a)$ is the action-value function that SARSA learns.

   This makes sense because SARSA is estimating action-values while following π, so once we know the expected values for all actions, we just take their weighted sum based on how often the policy chooses them.

   SARSA updates $Q^\pi(s,a)$ using the TD(0) update rule:

   $$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha\left[r_t + \gamma Q(s_{t+1},a_{t+1}) - Q(s_t,a_t)\right]$$

   Since this process gradually improves Q-values, we can confidently estimate $V^\pi(s)$ once the algorithm stabilizes.

2. **(10 points)** Does SARSA update equation change when we modify the reward notations ($R(s)$, $R(s,a)$, or $R(s,a,s')$). Why or why not?

   No, the SARSA update equation does not change when we modify the reward notation.

   The SARSA update rule remains:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]$$

where:

- s, a, r, s', a' represent the current state, action, received reward, next state, and next action.
- α is the learning rate, and γ is the discount factor.

The key reason why the update equation stays the same is that SARSA only uses the actual received reward r in each step. It does not rely on the underlying definition of the reward function itself.

Changing the reward notation ($R(s)$, $R(s,a)$, or $R(s,a,s')$) only affects how the environment assigns rewards, but not how SARSA processes them. Regardless of whether the reward depends on: - just the state ($R(s)$), - the state-action pair ($R(s,a)$), or - the full transition ($R(s,a,s')$),

SARSA always takes the observed r as input and updates $Q(s,a)$ accordingly. Since the update rule works the same way regardless of how the reward was generated, modifying the reward notation does not change the SARSA equation.

3. **(10 points)** Policy iteration algorithm consists of a policy evaluation phase and a policy improvement phase. If we replace dynamic programming policy evaluation with First Visit Monte Carlo prediction for policy evaluation, will policy iteration converge to an optimal policy in a finite number of iterations? Why or why not? Assume the agent has access to T and R.

   Policy iteration consists of two alternating steps:

   - **Policy Evaluation:** Computes the value function $V^\pi(s)$ for a given policy π.
   - **Policy Improvement:** Updates the policy using the newly estimated value function to derive a better policy.

   Standard Dynamic Programming (DP) Policy Evaluation

   - In standard policy iteration, Dynamic Programming (DP) is used for policy evaluation, solving the Bellman equation iteratively to compute $V^\pi(s)$ exactly.
   - Since DP directly uses the transition model T and reward function R, it guarantees finite-step convergence for each policy evaluation step and as a result, policy iteration with DP converges to the optimal policy in a finite number of steps.

   Replacing DP with First-Visit Monte Carlo (MC) If we replace DP-based policy evaluation with First-Visit Monte Carlo Prediction, the following changes happen:

   - Monte Carlo Estimates Are Sample-Based (Not Exact)
     - Unlike DP, Monte Carlo (MC) does not use the transition model T directly.
     - Instead, it estimates $V^\pi(s)$ using the average of sampled returns:

       $$V^\pi(s) = \frac{1}{N} \sum_{i=1}^{N} G_i, \quad G_t = \sum_{k=0}^{T} \gamma^k R_{t+k}$$

       where G_i is the observed return from state s.
     - Because MC estimates are based on finite samples, they fluctuate due to variance and only converge in expectation as $N \to \infty$.

- No Finite-Step Convergence Guarantee
  - Standard DP-based policy iteration ensures that each policy evaluation step completes in a finite number of iterations.
  - However, MC estimates depend on sample quality and number of episodes, leading to no deterministic convergence rate.
  - The number of iterations required for policy iteration to converge is not guaranteed to be finite when using MC evaluation.
- Delayed Updates & High Variance
  - MC updates $V^\pi(s)$ only after full episodes are completed, delaying feedback compared to DP.
  - Variance in MC estimates can lead to slow or unstable convergence, especially in early iterations.
  - If the policy doesn't explore all states sufficiently, MC estimates may never fully converge.

Replacing DP-based policy evaluation with First-Visit Monte Carlo removes the finite-step convergence guarantee of policy iteration. Since MC estimates only converge in expectation over infinite samples, policy iteration may take an indefinite number of iterations to reach the optimal policy. In contrast, DP ensures exact policy evaluation in a finite number of steps, making it the preferred approach when the transition model T and reward function R are known.

4. **(20 points)** Consider actor-critic methods in tabular format.
(i) We will replace the TD-critic module with that of an Oracle critic: the critic has an MDP model (unknown to the actor) and can calculate $V(s)$ accurately using the Bellman equation. How does this affect the actor updates and convergence of the algorithm?

- **Accurate Policy Updates:**
The Oracle critic computes the exact value function $V^\pi(s)$ using the Bellman equation:

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s\right].$$

In contrast, standard TD-learning updates the value function incrementally, introducing estimation noise and variance. The Oracle critic eliminates this error, leading to a zero-variance TD error:

$$\delta_t = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t).$$

The actor's policy gradient update now relies on an exact advantage estimate, improving stability:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \ln \pi(a_t \mid s_t) \cdot \delta_t\right].$$

With no bootstrapping error, updates are precise and stable.
- **Faster Convergence:**
In standard actor-critic, the TD-critic must learn $V^\pi(s)$ over multiple iterations. With an Oracle critic:
  - The actor instantly receives the true value function, eliminating critic learning.
  - Policy updates follow the true gradient direction, reducing the number of iterations required for convergence.
- **Zero Variance in Critic:**
Unlike TD-learning, which bootstraps from sampled transitions and introduces variance, the Oracle critic provides deterministic value estimates. This removes the need for variance reduction techniques like baselines or advantage normalization.

- **Limitations:**
  While beneficial, using an Oracle critic is not practical in most RL settings:
  - The Oracle critic assumes full knowledge of the MDP (transition model $T(s' \mid s, a)$ and reward function $R(s, a)$), which is rarely available.
  - If the critic's model is imperfect, the actor will optimize a biased value function, leading to suboptimal policies.
- **Convergence:**
  With an Oracle critic, the algorithm converges deterministically to the globally optimal policy, assuming:
  - The actor's learning rate α is properly tuned (not too large to cause oscillations).
  - The policy π is expressive enough to represent the optimal policy.
  - Sufficient exploration is ensured (e.g., using ϵ-greedy or softmax exploration).

Since the critic has perfect information, it eliminates bias and variance in value estimates, ensuring policy gradient updates follow the true gradient, unlike TD-critic, which relies on bootstrapped estimates.

(ii) We will now replace the TD-critic module with that of a lazy critic: it updates the values $V(s)$ in every other iteration (instead of every iteration). How does this affect the actor updates and convergence of the algorithm?

- **Stale Value Estimates:**
  Since the lazy critic updates $V(s)$ only every other iteration, the actor sometimes relies on outdated value estimates. This results in a biased TD error:

  $$\delta_t = r_t + \gamma V^{\text{old}}(s_{t+1}) - V^{\text{old}}(s_t).$$

  This affects the policy gradient update:

  $$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \ln \pi(a_t \mid s_t) \cdot \delta_t\right].$$

  Since δ_t is based on old values, policy updates can move in suboptimal directions.
- **Delayed Value Propagation and Slower Convergence:**
  Since the critic updates less frequently, value estimates take longer to propagate, which slows convergence because:
  - The actor's updates lag behind true value improvements, reducing learning efficiency.
  - Policy corrections happen less frequently, requiring more iterations to reach the optimal policy.
- **Potential Instability:**
  If the actor updates aggressively (e.g., with a high learning rate) while the critic lags, the policy may oscillate or diverge. This occurs because:
  - The actor's updates are not aligned with the true gradient direction.
  - Outdated $V(s)$ values can mislead the actor, causing overcorrections.
- **Bias-Variance Tradeoff:**
  Less frequent critic updates slightly reduce variance in $V(s)$, but this comes at the cost of increased bias. The overall effect is negative because:
  - The bias introduced by stale $V(s)$ dominates, making learning inefficient.
  - The reduced variance does not compensate for the slower convergence and instability.

- **Convergence:**
  The algorithm can still converge to a locally optimal policy, but:
  - Convergence is slower due to delayed critic updates.
  - The policy may stabilize at a suboptimal point before the critic can correct it.
  - Proper tuning of learning rates (for both actor and critic) is crucial to avoid instability.

While the lazy critic may reduce variance slightly, the increased bias from outdated values dominates, making learning inefficient and potentially leading to convergence to a suboptimal policy.

5. **(25 points)** Consider the following episodes of an MDP with five states and five actions. Each entry in the episode includes the state id, action id, and the corresponding reward observed. Discount factor $\gamma = 0.9$.
(i) For each episode, calculate the returns of each state using *first-visit Monte Carlo* algorithm and calculate the value of each state $V(s)$ using both the episodes.
(ii) For each episode, calculate the returns of each state using e*very-visit Monte Carlo* algorithm and calculate the value of each state $V(s)$ using both the episodes. Report your values as a table to facilitate faster grading.

- Episode 1: $\{(s_1, a_1, 1), (s_1, a_1, 5), (s_2, a_4, 8), (s_3, a_5, 5), (s_4, a_1, 10)\}$
- Episode 2: $\{(s_1, a_1, 5), (s_2, a_4, -1), (s_3, a_5, -2), (s_5, a_2, 1), (s_4, a_1, 10)\}$

For each time step t, the return G_t is computed using:

$$G_t = \sum_{k=0}^{T-t} \gamma^k R_{t+k}$$

where:

- R_{t+k} is the reward observed at step $t + k$.
- $\gamma = 0.9$ is the discount factor.
- T is the total number of steps in the episode.

Episode 1: Return Calculations

Step	State	G_t
0	s_1	$1 + (0.9)(5) + (0.9)^2(8) + (0.9)^3(5) + (0.9)^4(10) = 22.19$
1	s_1	$5 + (0.9)(8) + (0.9)^2(5) + (0.9)^3(10) = 23.54$
2	s_2	$8 + (0.9)(5) + (0.9)^2(10) = 20.6$
3	s_3	$5 + (0.9)(10) = 14$
4	s_4	10

Episode 2: Return Calculations

Step	State	G_t
0	s_1	$5 + (0.9)(-1) + (0.9)^2(-2) + (0.9)^3(1) + (0.9)^4(10) = 9.77$
1	s_2	$-1 + (0.9)(-2) + (0.9)^2(1) + (0.9)^3(10) = 5.3$
2	s_3	$-2 + (0.9)(1) + (0.9)^2(10) = 7$
3	s_5	$1 + (0.9)(10) = 10$
4	s_4	10

First-Visit Monte Carlo estimates $V(s)$ by averaging the first occurrence returns for each state:

$$V(s) = \frac{\sum G_i}{\text{number of first visits}}$$

State	G_i	$V(s)$
s_1	$\{22.19, 9.77\}$	$\frac{22.19+9.77}{2} = 15.98$
s_2	$\{20.6, 5.3\}$	$\frac{20.6+5.3}{2} = 12.95$
s_3	$\{14, 7\}$	$\frac{14+7}{2} = 10.5$
s_4	$\{10, 10\}$	$\frac{10+10}{2} = 10.0$
s_5	$\{10\}$	$\frac{10}{1} = 10.0$

Every-Visit Monte Carlo estimates $V(s)$ by averaging all occurrences of each state:

$$V(s) = \frac{\sum G_i}{\text{total occurrences of } s}$$

State	G_i	$V(s)$
s_1	$\{22.19, 23.54, 9.77\}$	$\frac{22.19+23.54+9.77}{3} = 18.49$
s_2	$\{20.6, 5.3\}$	$\frac{20.6+5.3}{2} = 12.95$
s_3	$\{14, 7\}$	$\frac{14+7}{2} = 10.5$
s_4	$\{10, 10\}$	$\frac{10+10}{2} = 10.0$
s_5	$\{10\}$	$\frac{10}{1} = 10.0$

First-Visit Monte Carlo only counts the first occurrence of each state in an episode. and every-Visit Monte Carlo averages all occurrences of each state, making it more stable. The values computed using Every-Visit MC are typically more robust since more samples are used.

6. **(25 points)** Consider two MDPs $\mathcal{M} = \langle S, A, T, R \rangle$ and $\mathcal{M}' = \langle S, A, T, R' \rangle$. The MDPs have the same state space, action space, transition functions, and discuount factor γ but differ in their reward functions such that $R'(s, a, s') = R(s, a, s') + F(s, a, s')$ where $F(s, a, s')$ is a bonus reward that could help speed up the learning process (also referred to as reward shaping). In our case, $F(s, a, s') = \gamma\phi(s') - \phi(s)$ for some arbitrary function $\phi : S \to \mathbb{R}$.

   Consider running tabular Q-learning in each MDP $\mathcal{M}$ and $\mathcal{M}'$, with initial values $Q^0_{\mathcal{M}}(s, a) = q_{\text{init}} + \phi(s)$ and $Q^0_{\mathcal{M}'}(s, a) = q_{\text{init}}$, $\forall (s, a) \in S \times A$ and $q_{\text{init}} \in \mathbb{R}$. At any moment in time, the current Q-value of any state-action pair is always equal to its initial value plus some Δ value denoting the total change in the Q-value across all updates:

$$Q_{\mathcal{M}}(s, a) = Q^0_{\mathcal{M}}(s, a) + \Delta Q_{\mathcal{M}}(s, a) \tag{1}$$
$$Q_{\mathcal{M}'}(s, a) = Q^0_{\mathcal{M}'}(s, a) + \Delta Q_{\mathcal{M}'}(s, a) \tag{2}$$

   Show that if $\Delta Q_{\mathcal{M}}(s, a) = \Delta Q_{\mathcal{M}'}(s, a)$ for all $(s, a) \in S \times A$, then these two Q-learning agents yield identical updates for any state-action pair. That is, both agents will converge to policies that behave identically, and the offset $\phi(s)$ will not affect action selection.

   Hints:

- Equations (1) and (2) indicate that $\Delta Q_{\mathcal{M}}(s,a) = \alpha_M \delta_M$ and $\Delta Q_{\mathcal{M}'}(s,a) = \alpha_{M'} \delta_{M'}$ where α_M and $\alpha_{M'}$ denote the learning rates or step sizes and δ_M and $\delta_{M'}$ denotes the corresponding TD-errors.
- For ease, you can assume that $\alpha_M = \alpha_{M'}$.
- What you then need to show is $\delta_M = \delta_{M'}$. To do so, use Equations (1) and (2) to rewrite $Q_{\mathcal{M}}(s,a)$ and r_M in terms of $Q_{\mathcal{M}'}(s,a)$ and $r_{M'}$ or vice versa.

Given,

$$R'(s,a,s') = R(s,a,s') + F(s,a,s'),$$

where the shaping function is given by:

$$F(s,a,s') = \gamma\phi(s') - \phi(s),$$

for some function $\phi : S \to \mathbb{R}$.

We run tabular Q-learning on both MDPs, with the initial conditions:

$$Q^0_{\mathcal{M}}(s,a) = q_{\text{init}} + \phi(s), \quad Q^0_{\mathcal{M}'}(s,a) = q_{\text{init}}, \quad \forall (s,a) \in S \times A.$$

At any time step, the Q-values evolve as:

$$Q_{\mathcal{M}}(s,a) = Q^0_{\mathcal{M}}(s,a) + \Delta Q_{\mathcal{M}}(s,a),$$

$$Q_{\mathcal{M}'}(s,a) = Q^0_{\mathcal{M}'}(s,a) + \Delta Q_{\mathcal{M}'}(s,a).$$

$$\Delta Q_{\mathcal{M}}(s,a) = \Delta Q_{\mathcal{M}'}(s,a), \quad \forall (s,a) \in S \times A.$$

To prove that if this assumption holds, both Q-learning agents will have identical updates at every step and will converge to the same policy.

To do so, we need to prove that the TD-errors are equal, i.e.,

$$\delta_{\mathcal{M}} = \delta_{\mathcal{M}'}.$$

First we need to express TD Errors in Terms of Each Other and the Q-learning TD error is defined as:

$$\delta = r + \gamma \max_{a'} Q(s',a') - Q(s,a).$$

For MDP $\mathcal{M}$, we write:

$$\delta_{\mathcal{M}} = R(s,a,s') + \gamma \max_{a'} Q_{\mathcal{M}}(s',a') - Q_{\mathcal{M}}(s,a).$$

For MDP $\mathcal{M}'$, substituting $R'(s,a,s') = R(s,a,s') + \gamma\phi(s') - \phi(s)$, we get:

$$\delta_{\mathcal{M}'} = (R(s,a,s') + \gamma\phi(s') - \phi(s)) + \gamma \max_{a'} Q_{\mathcal{M}'}(s',a') - Q_{\mathcal{M}'}(s,a).$$

$$Q_{\mathcal{M}}(s,a) = Q_{\mathcal{M}'}(s,a) + \phi(s).$$

$$\max_{a'} Q_{\mathcal{M}}(s',a') = \max_{a'}(Q_{\mathcal{M}'}(s',a') + \phi(s')).$$

Since $\phi(s')$ is independent of action selection, it factors out:

$$\max_{a'} Q_{\mathcal{M}}(s',a') = \max_{a'} Q_{\mathcal{M}'}(s',a') + \phi(s').$$

Substituting this into $\delta_{\mathcal{M}}$:

$$\delta_{\mathcal{M}} = R(s,a,s') + \gamma(\max_{a'} Q_{\mathcal{M}'}(s',a') + \phi(s')) - (Q_{\mathcal{M}'}(s,a) + \phi(s)).$$

Expanding:

$$\delta_{\mathcal{M}} = \left(R(s,a,s') + \gamma \max_{a'} Q_{\mathcal{M}'}(s',a') - Q_{\mathcal{M}'}(s,a)\right) + \gamma\phi(s') - \phi(s).$$

$$\delta_{\mathcal{M}} = \delta_{\mathcal{M}'} + \gamma\phi(s') - \phi(s).$$

To prove TD Errors are Equal

$$\Delta Q_{\mathcal{M}}(s,a) = \Delta Q_{\mathcal{M}'}(s,a).$$

Since the Q-learning update follows:

$$\Delta Q_{\mathcal{M}}(s,a) = \alpha\delta_{\mathcal{M}}, \quad \Delta Q_{\mathcal{M}'}(s,a) = \alpha\delta_{\mathcal{M}'}.$$

Equating both,

$$\alpha\delta_{\mathcal{M}} = \alpha\delta_{\mathcal{M}'}.$$

Since α is the same for both MDPs, we obtain:

$$\delta_{\mathcal{M}} = \delta_{\mathcal{M}'}.$$

Thus, the learning updates in both MDPs remain identical at every step.

To show that Both Agents Converge to the Same Policy

Q-learning chooses actions based on:

$$\pi(s) = \arg\max_{a} Q(s,a).$$

$$Q_{\mathcal{M}}(s,a) = Q_{\mathcal{M}'}(s,a) + \phi(s),$$

and $\phi(s)$ is independent of action selection, it follows that:

$$\arg\max_a Q_{\mathcal{M}}(s,a) = \arg\max_a Q_{\mathcal{M}'}(s,a).$$

This means that both Q-learning agents will select the same actions at every step and will converge to the same policy.

The function $\phi(s)$ shifts all Q-values by a constant, but this does not affect the learning updates. Since the TD error remains the same in both cases, the learning process proceeds identically in both MDPs. Action selection is based on the ranking of Q-values, and since $\phi(s)$ is state-dependent but not action-dependent, it does not alter the optimal action chosen in any state. As a result, both Q-learning agents will behave identically despite the initial difference in reward structures.